

Arquivo: node_registry.py

```
"""
DESCRIÇÃO GERAL:
Este módulo centraliza a lista de nós do cluster.
Ele funciona como um cadastro dos nós existentes, facilitando a descoberta dos
endereços, portas e diretórios de cada nó.
"""

from dataclasses import dataclass
from pathlib import Path

from dfs.config import NODES, NODE_ORDER

@dataclass(frozen=True)
class NodeInfo:
    """
    Estrutura simples para guardar os dados de um nó.
    """
    node_id: str
    host: str
    port: int
    storage_dir: Path

class NodeRegistry:
    """
    Mantém e organiza os nós do cluster.
    """

    def __init__(self, nodes: dict[str, dict] | None = None):
        # Se não receber configuração externa, usa a configuração global.
        raw_nodes = nodes or NODES

        # Converte o dicionário bruto em objetos NodeInfo.
        self._nodes: dict[str, NodeInfo] = {
            node_id: NodeInfo(
                node_id=node_id,
                host=spec["host"],
                port=spec["port"],
                storage_dir=Path(spec["storage_dir"]),
            )
            for node_id, spec in raw_nodes.items()
        }

        # Mantém a ordem dos nós definida em config.py.
        self._ordered_ids = [node_id for node_id in NODE_ORDER if node_id in self._nodes]

        # Caso algum cenário de teste altere os nós, garante uma ordem mínima válida.
        if not self._ordered_ids:
            self._ordered_ids = list(self._nodes.keys())
```

```
def list_nodes(self) -> list[NodeInfo]:
    """
    Retorna todos os nós conhecidos.
    """
    return [self._nodes[node_id] for node_id in self._ordered_ids]

def get(self, node_id: str) -> NodeInfo:
    """
    Retorna as informações de um nó específico.
    """
    return self._nodes[node_id]

def get_by_index(self, index: int) -> NodeInfo:
    """
    Retorna um nó pela posição na ordem definida.
    """
    node_id = self._ordered_ids[index % len(self._ordered_ids)]
    return self._nodes[node_id]

def index_of(self, node_id: str) -> int:
    """
    Retorna o índice de um nó na ordem do cluster.
    """
    return self._ordered_ids.index(node_id)

def size(self) -> int:
    """
    Retorna a quantidade total de nós.
    """
    return len(self._ordered_ids)
```